ORIGINAL RESEARCH - PILOT STUDY

Beyond Big-O: A Reproducible Apple-Silicon Pilot Study of Runtime Performance for Common Data Structures in Python and C++

Mahesh Patil and Student Research Team

Affiliation to be added before submission

Draft status: Prepared from measurements executed in Codex on 23 August 2026, with supplementary construct-validity (Section 4.5) and third-language (Section 4.6) checks added on 25 August 2026. This manuscript is intended for internal academic review and methodological refinement before journal submission.

Abstract

Asymptotic analysis predicts how algorithmic cost grows, but it does not capture language-runtime overhead, container representation, allocation behaviour or constants that influence observed execution time. This pilot study compared dynamic arrays, linked structures and hash tables under identical build, search, deletion and traversal workloads in Python 3.13 and optimized C++17. Experiments were executed on an Apple M2 MacBook Air with 8 GB memory and macOS 26.5.2. Four input sizes (1,000-25,000) were tested using three warm-ups and ten recorded repetitions, yielding 240 validated measurement records. At n = 25,000, median Python runtimes were 1.86-91.68 times the corresponding C++ medians across the twelve structure-operation combinations; the upper end of this range reflects a submicrosecond, resolution-limited C++ traversal median (Section 3.4) rather than a stable absolute figure. Batched search and deletion for sequential structures exhibited empirical scaling exponents of approximately 1.85-2.00 because both the collection size and the number of operations increased with n; hash workloads were closer to linear. All implementations produced identical search-hit counts and post-deletion checksums, including a supplementary re-run of the linked benchmark using a singly-linked std::forward_list to match the Python implementation's representation, which reproduced the same pattern of ratios (Section 4.5) rather than narrowing it. A further supplementary run added Java as a third language (Section 4.6): Java's ratio to C++ was operation-dependent rather than uniformly intermediate between Python and C++, ranging from 0.41x (Java faster than C++ for linked insertion) to 109.78x (array traversal), and Java was slower than Python on two of the twelve structure-operation combinations despite being faster overall. The findings support teaching Big-O together with implementation-aware measurement, while the single-machine scope, absence of

hardware-counter data, and submicrosecond C++ measurements make the work a pilot rather than a final general-purpose benchmark.

Keywords: data structures; Big-O; empirical algorithmics; Python; C++; Java; microbenchmarking; reproducibility; construct validity

1. Introduction

Big-O notation provides a machine-independent description of growth as input size increases. It is indispensable for algorithm design, yet it intentionally suppresses constant factors, representation costs and hardware effects [1]. Consequently, two implementations with the same asymptotic class can differ substantially in observed runtime, while a theoretically favourable structure may carry a higher construction or traversal cost for a particular workload.

This distinction is especially important in undergraduate computing education. Students often learn that hash-table lookup is expected O(1), array search is O(n), and linked-list traversal is O(n), but they may not observe how language runtimes, boxed objects, allocation strategies and contiguous memory influence actual measurements. Computer architecture texts likewise emphasise that locality and the memory hierarchy shape performance beyond instruction counts [2].

The objective of this pilot is not to declare one programming language universally superior. It is to test whether a small, reproducible experiment can connect theoretical complexity with measured behaviour while making its assumptions and limitations explicit. The resulting protocol is intended as a foundation for a student research project and a later multi-platform study.

1.1 Research questions

- RQ1: How do language and data-structure implementation affect observed runtime for equivalent workloads?
- RQ2: Do measured scaling patterns agree with the workload-level complexity predicted from Big-O analysis?
- RQ3: Which methodological limitations must be addressed before extending the pilot into a journal-ready benchmark?

1.2 Contributions

- A common, deterministic dataset and workload definition shared by Python and C++ implementations.
- A reproducible execution pipeline with correctness checks, raw-data hashing and environment metadata.

- An empirical distinction between per-operation complexity and the complexity of a batch whose operation count also grows with n.
- A transparent account of measurement-resolution, implementation-equivalence and external-validity limitations.
- An empirical test of whether the C++/Python linked-container mismatch inflates the observed language gap, rather than leaving that concern as an unverified caveat.
- A third-language check that re-runs the full workload under Java, testing whether the Python/C++ pattern generalises rather than leaving Java's absence as an unaddressed limitation.

2. Background and Related Work

The analysis of algorithms separates growth rate from implementation detail [1]. Experimental algorithmics complements that abstraction by examining implementations on specified workloads and machines. Algorithm engineering has been described as a methodology connecting design, implementation, experimentation and refinement [3]. This perspective motivates reporting sufficient detail to reproduce not only the code but also the experimental conditions.

Performance measurement is vulnerable to effects that are easy to overlook. Mytkowicz et al. showed that apparently harmless environmental changes can alter conclusions [4]. Georges et al. and Kalibera and Jones argued for warm-ups, repeated measurements, uncertainty reporting and explicit treatment of runtime-system variability [5,6]. Arcuri and Briand outlined complementary statistical-testing practice for evaluating randomized algorithms in software engineering, including effect-size reporting alongside significance testing [7]. Fleming and Wallace further cautioned that benchmark summaries can mislead when inappropriate averages are used [8]. Reproducibility itself depends on preserving code, data and environment details rather than reporting summary numbers alone [9,10]. The present pilot therefore prioritises medians, dispersion, warm-ups, immutable raw results and correctness checks.

The study is deliberately modest. It does not claim that its three containers exhaust data-structure design or that Python and C++ expose equivalent internal representations. Instead, it investigates idiomatic implementations under a shared external workload. That decision improves classroom relevance but weakens causal attribution: measured differences reflect the combined effect of language, runtime, library implementation and element representation.

3. Materials and Methods

3.1 Execution environment

Table 1. Recorded execution environment.

Item	Recorded value
Computer	MacBook Air (Mac14,2)
Processor	Apple M2, 8 cores (4 performance, 4 efficiency), arm64
Memory	8 GB unified memory
Operating system	macOS 26.5.2, Darwin 25.5.0
Python	CPython 3.13.5
C++	C++17 compiled with Apple Clang 21.0.0 using -O2
Java	Not executed: no working JDK was available
Hardware counters	Not collected: Linux perf unavailable on macOS
Execution date	23 August 2026

3.2 Structures and workload

The experiment used three abstract structures. The dynamic-array condition used Python list [11] and C++ std::vector<int> [12]; the linked condition used a custom Python singly linked list and C++ std::list<int>; and the hash condition used Python dict and C++ std::unordered_map<int,int>. Inputs contained unique integers in a deterministic shuffled order.

Table 2. Common experimental workload.

Component	Definition
Input size	1,000; 5,000; 10,000; 25,000
Build	Construct the structure from all n values
Search	Query max(10, 0.01n) keys; half present and half absent
Delete	Delete max(1, 0.01n) keys known to be present
Traverse	Sum all keys remaining after deletion
Correctness	Compare search-hit count and post-deletion checksum

Component	Definition
Timing boundary	Exclude dataset parsing and CSV output

3.3 Dataset generation and validation

A fixed seed (20260823) generated one text dataset per input size. Every language read the same files. A manifest stored each file's SHA-256 digest. Before performance analysis, all result groups were checked for identical search-hit counts and identical post-deletion checksums. Any disagreement would have invalidated the performance data; none occurred.

3.4 Measurement protocol

For every language-structure-size combination, the program performed three unrecorded warm-ups followed by ten recorded repetitions. Jobs were shuffled with the experiment seed. Operation durations were measured using monotonic high-resolution clocks and reported in nanoseconds. The complete design produced 2 languages x 3 structures x 4 sizes x 10 repetitions = 240 records.

> **Measurement caveat:** Some optimized C++ workloads completed in less than one microsecond, and the C++ hash-search median at n = 1,000 was recorded as zero. Such values are below a dependable single-shot timing range and are treated as resolution-limited rather than literal zero-cost operations.

3.5 Analysis

The primary summary was the median across ten repetitions, with the mean, standard deviation and non-parametric bootstrap interval retained in the generated analysis file [13]. Coefficient of variation (CV) was used to describe group dispersion. Python-to-C++ median ratios at n = 25,000 were calculated descriptively; Cliff's delta [14], following the A-measure formulation of Vargha and Delaney [15], was also computed. Scaling exponents between n = 5,000 and n = 25,000 were estimated as log(T2/T1) / log(n2/n1). No causal language claim is made because language and container representation were not independently manipulated.

4. Results

4.1 Correctness and measurement stability

All 240 records passed the hit-count and checksum validation. The median CV across the 96 language-structure-size-operation groups was 4.0%. The maximum CV was 129.1% and occurred in the resolution-limited C++ hash-search group at n = 1,000. The data therefore show generally stable repeated measurements while also identifying the exact region in which the timer cannot support strong absolute claims.

4.2 Search scaling

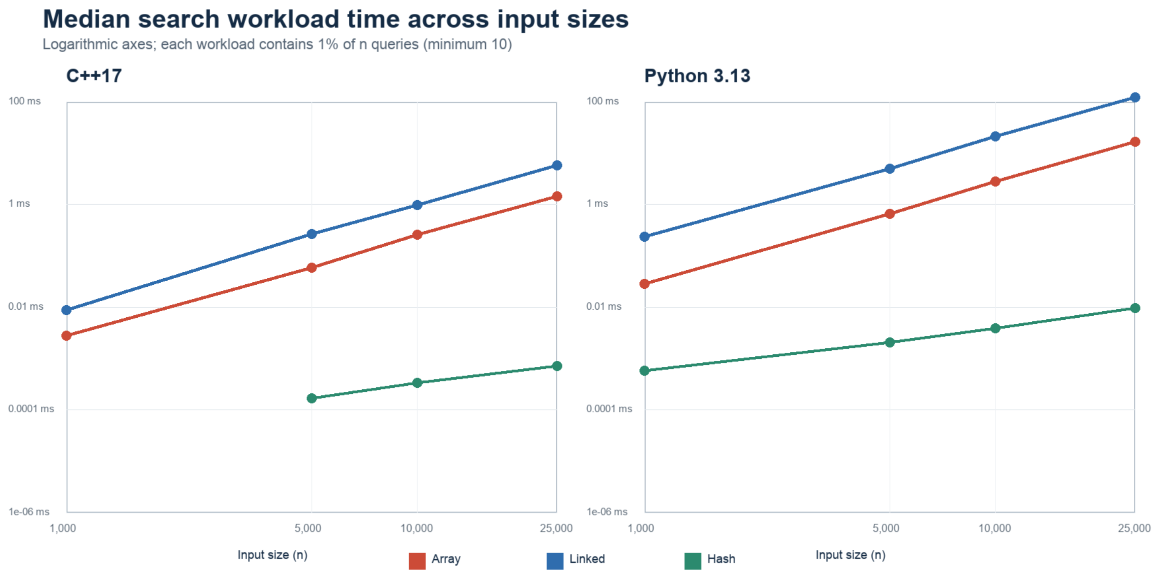

Figure 1. Median search-workload time across input sizes. The zero C++ hash-search median at n = 1,000 is omitted from the logarithmic plot.

Sequential search rose much more sharply than hash search. Between 5,000 and 25,000 elements, the estimated search exponent was 1.99 for the C++ array and 2.00 for the Python array; the linked implementations produced 1.91 and 1.99, respectively. The corresponding hash exponents were 0.90 for C++ and 0.97 for Python. This apparent quadratic-versus-linear contrast is consistent with the workload definition: the number of search queries itself increases in proportion to n, so an O(n) sequential search repeated O(n) times yields an O(n^2) batch, whereas expected O(1) hash lookup repeated O(n) times yields an O(n) batch.

4.3 Runtime at the largest input

Table 3. Median workload runtime at n = 25,000 (ten repetitions).

Structure	Operation	C++17	Python 3.13	Python/C++
Array	Insert	0.0066 ms	0.0312 ms	4.76x
Array	Search	1.425 ms	16.389 ms	11.50x
Array	Delete	1.072 ms	16.401 ms	15.30x

Structure	Operation	C++17	Python 3.13	Python/C++
Array	Traverse	0.958 us	0.0878 ms	91.68x
Linked	Insert	0.2152 ms	3.376 ms	15.69x
Linked	Search	5.712 ms	123.330 ms	21.59x
Linked	Delete	3.742 ms	76.788 ms	20.52x
Linked	Traverse	0.0297 ms	0.7406 ms	24.91x
Hash	Insert	0.2779 ms	0.6200 ms	2.23x
Hash	Search	0.708 us	0.0096 ms	13.57x
Hash	Delete	0.0049 ms	0.0091 ms	1.86x
Hash	Traverse	0.0295 ms	0.1087 ms	3.68x

At n = 25,000, every Python median exceeded its corresponding C++ median; the descriptive ratios ranged from 1.86x for hash deletion to 91.68x for array traversal. Cliff's delta was 1.00 for all twelve comparisons, meaning every recorded Python duration in each comparison exceeded every corresponding C++ duration. These values characterise the tested implementations on this Mac and must not be interpreted as universal language speed factors.

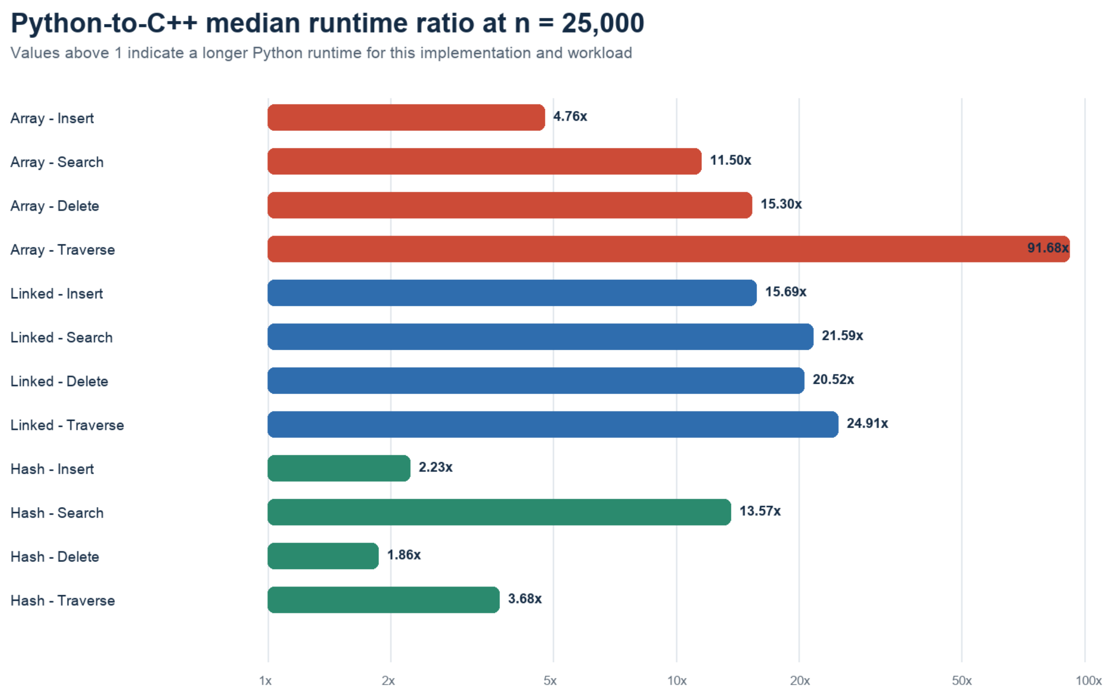

Figure 2. Python-to-C++ median runtime ratios at n = 25,000. The horizontal axis is logarithmic.

4.4 Deletion and traversal patterns

Array and linked deletion workloads also approached quadratic scaling because 1% of n keys were deleted and each deletion required sequential location; estimated exponents ranged from 1.85 to 1.89 for linked deletion and 1.87 to 1.89 for array deletion. Hash deletion remained close to linear at 1.00 in C++ and 1.02 in Python. Traversal generally approached linear growth, with exponents between 0.84 and 0.97 across the structures and languages. These patterns are more defensible than the smallest absolute timings because they rely on changes across larger workloads.

4.5 Construct-validity check: a singly-linked C++ comparison

Section 6.2 notes a representational mismatch in the linked condition: the C++ side used std::list, which is typically doubly linked, while the Python side used a custom singly linked list. To test whether this mismatch inflates the reported ratios rather than merely being noted as a caveat, the C++ linked benchmark was re-run using std::forward_list, a singly-linked container with the same single-next-pointer structure as the Python implementation, across all four input sizes with the same three-warm-up, ten-repetition protocol. Every forward_list run reproduced the same search-hit count and post-deletion checksum as both the original std::list run and the Python run at every input size, verified programmatically before the comparison was computed.

Table 4. Construct-validity check: median linked-structure runtime at n = 25,000 using a singly-linked C++ container matched to the Python implementation.

Operation	C++ std::list	C++ std::forward_list	Python 3.13	Python/forward_list
Insert	0.2152 ms	0.2801 ms	3.376 ms	12.05x
Search	5.712 ms	4.670 ms	123.330 ms	26.41x
Delete	3.742 ms	3.232 ms	76.788 ms	23.76x
Traverse	0.0297 ms	0.0231 ms	0.7406 ms	32.05x

Matching the C++ container's linkage discipline to the Python implementation did not narrow the measured language gap. The Python-to-C++ ratio was higher under the forward_list comparison than under std::list for search (26.41x versus 21.59x), deletion (23.76x versus 20.52x) and traversal (32.05x versus 24.91x), and only slightly lower for insertion (12.05x versus 15.69x). Between n = 5,000 and n = 25,000, the forward_list search and deletion exponents were 1.78 and 1.64 respectively, consistent with the batch-scaling explanation given in Section 5.1 for std::list. The construct-validity concern raised in Section 6.2 therefore does not appear to be inflating the reported Python/C++ ratios for the linked condition, though it remains a caveat for claims about a specific container's internal performance rather than about Python versus C++ generally.

4.6 External-validity check: adding Java as a third language

Section 6.3 notes that Java was not executed in the primary study because no working JDK was available at measurement time. To test whether the reported Python/C++ pattern generalises to a third, managed-runtime language, the Java benchmark already scaffolded in the repository was compiled and run under OpenJDK 26.0.2.1 (Homebrew) across all three structures and four input sizes with the identical three-warm-up, ten-repetition protocol, on 25 August 2026. Every Java run reproduced the same search-hit count and post-deletion checksum as the corresponding Python and C++ runs at every input size, verified programmatically before the comparison was computed.

Table 5. Median workload runtime at n = 25,000 including Java (ten repetitions).

Structure	Operation	C++17	Python 3.13	Java	Java/C++
Array	Insert	0.0066 ms	0.0312 ms	0.2406 ms	36.66x
Array	Search	1.425 ms	16.389 ms	2.121 ms	1.49x
Array	Delete	1.072 ms	16.401 ms	1.732 ms	1.62x
Array	Traverse	0.958 us	0.0878 ms	0.1052 ms	109.78x

Structure	Operation	C++17	Python 3.13	Java	Java/C++
Linked	Insert	0.2152 ms	3.376 ms	0.0890 ms	0.41x
Linked	Search	5.712 ms	123.330 ms	9.382 ms	1.64x
Linked	Delete	3.742 ms	76.788 ms	6.096 ms	1.63x
Linked	Traverse	0.0297 ms	0.7406 ms	0.0491 ms	1.65x
Hash	Insert	0.2779 ms	0.6200 ms	0.5437 ms	1.96x
Hash	Search	0.708 us	0.0096 ms	0.0086 ms	12.09x
Hash	Delete	0.0049 ms	0.0091 ms	0.0149 ms	3.03x
Hash	Traverse	0.0295 ms	0.1087 ms	0.4498 ms	15.25x

Java's ratio to C++ was operation-dependent rather than uniformly intermediate between the interpreted and compiled extremes: across the twelve structure-operation combinations it ranged from 0.41x (linked insertion, where Java's median was faster than C++'s) to 109.78x (array traversal), a wider spread than the Python/C++ range for the same twelve combinations. Two results are worth flagging specifically because they run against the intuition that a JIT-compiled language should sit strictly between interpreted Python and compiled C++. First, Java's array-traversal median (0.105 ms) was slower than Python's (0.088 ms) despite both being far above C++'s submicrosecond median, plausibly reflecting boxed-Integer ArrayList iteration overhead that a primitive int[] traversal would avoid. Second, Java's hash-traversal median (0.450 ms) was roughly four times slower than Python's (0.109 ms), even though Java outperformed Python on hash search and delete by a comparable margin in the other direction, consistent with boxed-Long iteration cost in the reference HashMap implementation rather than a general JVM weakness. Scaling exponents for Java search and deletion between n = 5,000 and n = 25,000 were 2.08 and 1.91 for the array and 1.97 and 1.85 for the linked structure, consistent with the same batch-scaling explanation given in Section 5.1. These results reinforce rather than complicate the paper's central methodological point: coarse language-level generalisations do not hold uniformly across operations, and only workload-level measurement exposes where they break down.

5. Discussion

5.1 Big-O explained, not contradicted

The results do not show a failure of asymptotic analysis. Instead, they demonstrate why the unit of analysis must be stated carefully. A single sequential search is O(n), but this experiment schedules 0.01n searches, creating an O(n^2) batch. A hash lookup is expected O(1) on average, and the corresponding

O(n) batch grew close to linearly. The empirical exponents therefore connect measured behaviour to the composed workload rather than replacing theory.

5.2 Language and representation effects

The Python/C++ differences include many inseparable factors: interpretation versus optimized native code, boxed Python integers versus C++ primitive integers, object allocation, iterator implementation and library design. The largest ratio occurred for contiguous traversal, where optimized native loops can exploit compact primitive storage, spatial locality and compiler transformations that pointer-chasing and boxed representations largely defeat [16,17]. Hash insertion and deletion showed smaller ratios, indicating that language overhead is not a single constant applied uniformly to every structure and operation.

5.3 Educational value

A useful teaching sequence is therefore: predict complexity, define the entire workload, implement correctness checks, measure, and then explain both agreement and disagreement. Students should learn to ask not only 'What is the Big-O?' but also 'What exactly is repeated, what representation is used, what is inside the timed region, and is the measurement long enough for the clock?' This echoes earlier calls in computing education to pair algorithmic analysis with empirical measurement rather than treating asymptotic notation as a substitute for it [18]. The repository makes those questions visible and reproducible.

6. Threats to Validity

6.1 Internal validity

Background activity, thermal conditions and operating-system scheduling were not instrumented. Although jobs were shuffled and measurements were repeated, the study did not pin processes to performance cores. Extremely short C++ operations are vulnerable to timer quantisation and clock-call overhead. A revised benchmark should batch pure operations until each timed interval exceeds a predefined duration and should separately quantify timing overhead.

6.2 Construct validity

The structures are functionally comparable but not internally identical. Python's custom singly linked list differs from C++ std::list, which is typically doubly linked. Python list holds object references, whereas std::vector<int> stores primitive integers contiguously. Consequently, the experiment evaluates idiomatic implementation conditions rather than isolating a pure language effect. This concern is tested empirically in Section 4.5, which reruns the C++ linked benchmark with the singly-linked std::forward_list; the resulting ratios do not shrink under the more closely matched comparison, indicating the linkage mismatch is not the source of the observed language gap. Peak memory and cache behaviour were proposed in the broader project but were not measured on this Mac and are not inferred in this paper.

6.3 External and conclusion validity

One Apple M2 laptop, one Python version and one C++ toolchain cannot represent other processors, operating systems, compilers or runtime configurations. Only four sizes and one random-input family were examined. Ten repetitions support descriptive stability but not broad population inference. Java was not executed in the primary 23 August 2026 session because no working JDK was available; Section 4.6 reports a supplementary Java run added on 25 August 2026 using a subsequently installed JDK on the same machine, so that check still shares this section's single-machine limitation. The conclusions are therefore restricted to this environment and protocol.

7. Conclusion and Future Work

This reproducible Mac pilot showed that theoretical growth and observed runtime can be taught together. Sequential search and deletion batches grew close to quadratically when both collection size and operation count increased, while hash batches grew close to linearly. Python and C++ showed substantial but operation-dependent runtime differences, and all correctness outputs agreed. A supplementary Java run showed that a JIT-compiled, managed-runtime language does not sit uniformly between the interpreted and compiled extremes: it was faster than C++ for linked insertion and slower than Python for two of the twelve structure-operation combinations. The study also exposed a measurement weakness: several optimized C++ workloads were too short for reliable single-shot timing.

Before journal submission, the benchmark should use calibrated operation batching, collect peak resident memory and hardware cache counters on a controlled Linux machine, record temperature and power mode, test additional input distributions, and reproduce the experiment on at least one independent system. The Java results in Section 4.6 address the language-coverage gap but not the single-machine, single-session limitation; a full three-language run alongside a second machine remains future work. Those extensions should be reported as new evidence, not combined silently with the present Mac data.

Data and Code Availability

The repository contains the dataset generator, Python, C++ and Java implementations, validation tests, execution configuration, raw CSV records and analysis scripts. The raw combined CSV contains 240 records and has SHA-256 digest c86df6732b7e5e78e7a71d6b72ae65535b648b10ccb4c39255a4652018ed3dff. The supplementary singly-linked construct-validity check (Section 4.5) adds 40 further C++ records with SHA-256 digest ccffc4c004456ee9474ce36489e7bb7c6dacb9bb2b3dfb96ba6e0108c1965bf8, and the supplementary Java run (Section 4.6) adds 120 further records with SHA-256 digest a925f8e56720fb234324b4bed00328c1643feb8e741eb44e9f7c1cc3ec705ea2. The repository is publicly available at https://github.com/maheshchudaman/beyond-big-o-research and is archived on Zenodo with concept DOI 10.5281/zenodo.22089928 (version-specific DOI for this release: 10.5281/zenodo.22089929).

Ethics, Authorship and AI-Assistance Statement

No human-participant, personal or sensitive data were used. Codex assisted with repository scaffolding, code review, execution orchestration, analysis scripting and preparation of this draft. All numerical claims in the manuscript were generated programmatically from the preserved raw CSV; missing measurements were not fabricated. Before submission, the named authors must independently verify the code, results, citations and wording, approve authorship contributions, and adapt this disclosure to the selected journal's policy.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
2. Hennessy, J. L., and Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
3. Sanders, P. (2009). Algorithm Engineering - An Attempt at a Definition. In Efficient Algorithms, LNCS 5760, 321-340. https://doi.org/10.1007/978-3-642-03456-5_22
4. Mytkowicz, T., Diwan, A., Hauswirth, M., and Sweeney, P. F. (2009). Producing wrong data without doing anything obviously wrong! Proceedings of ASPLOS XIV, 265-276. https://doi.org/10.1145/1508244.1508275
5. Georges, A., Buytaert, D., and Eeckhout, L. (2007). Statistically rigorous Java performance evaluation. Proceedings of OOPSLA 2007, 57-76. https://doi.org/10.1145/1297027.1297033
6. Kalibera, T., and Jones, R. (2013). Rigorous benchmarking in reasonable time. Proceedings of ISMM 2013, 63-74. https://doi.org/10.1145/2464157.2464160
7. Arcuri, A., and Briand, L. (2011). A practical guide for using statistical tests to assess randomized algorithms in software engineering. Proceedings of ICSE 2011, 1-10. https://doi.org/10.1145/1985793.1985795

8. Fleming, P. J., and Wallace, J. J. (1986). How not to lie with statistics: the correct way to summarize benchmark results. Communications of the ACM, 29(3), 218-221. https://doi.org/10.1145/5666.5673
9. Sandve, G. K., Nekrutenko, A., Taylor, J., and Hovig, E. (2013). Ten simple rules for reproducible computational research. PLOS Computational Biology, 9(10), e1003285. https://doi.org/10.1371/journal.pcbi.1003285
10. Peng, R. D. (2011). Reproducible research in computational science. Science, 334(6060), 1226-1227. https://doi.org/10.1126/science.1213847
11. Python Software Foundation. (2026). Python 3.13 documentation: Data structures. https://docs.python.org/3.13/tutorial/datastructures.html
12. ISO/IEC. (2020). ISO/IEC 14882:2020 Programming Languages - C++. International Organization for Standardization.
13. Efron, B., and Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman and Hall/CRC.
14. Cliff, N. (1993). Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological Bulletin, 114(3), 494-509. https://doi.org/10.1037/0033-2909.114.3.494
15. Vargha, A., and Delaney, H. D. (2000). A critique and improvement of the CL common language effect size statistics of McGraw and Wong. Journal of Educational and Behavioral Statistics, 25(2), 101-132. https://doi.org/10.3102/10769986025002101
16. Chilimbi, T. M., Hill, M. D., and Larus, J. R. (1999). Cache-conscious structure layout. Proceedings of PLDI 1999, 1-12. https://doi.org/10.1145/301618.301633
17. Drepper, U. (2007). What Every Programmer Should Know About Memory. Red Hat, Inc.
18. Astrachan, O. (2003). Bubble sort: an archaeological algorithmic analysis. ACM SIGCSE Bulletin, 35(1), 1-5. https://doi.org/10.1145/611892.611918

Appendix A. Reproducibility Record

Table A1. Repository evidence map.

Evidence	Repository path
Experiment configuration	config/experiment.json
Dataset manifest	data/generated/manifest.json
Raw measurements	results/raw/combined.csv
Environment metadata	results/raw/environment.json
Paper analysis	paper/analyse_for_paper.py
Correctness tests	tests/test_benchmark.py; tests/test_dataset_manifest.py
Construct-validity supplement (raw)	results/raw/cpp_linked_fwd_combined.csv
Construct-validity supplement (source)	src/cpp/benchmark.cpp (run_linked_fwd)
Java supplement (raw)	results/raw/java_combined.csv
Java supplement (source)	src/java/Benchmark.java

Evidence	Repository path